

EXPERIMENT - 3

Wireshark: Network Packet Capture and Analysis Tool

Procedure

Step 1: Start Capturing Packets

First, open Wireshark. You will see a list of all available network interfaces (e.g., “Wi-Fi,” “Ethernet,” “Loopback: lo”). Select the interface your computer is using to connect to the target (Loopback: lo, since the login page was hosted locally on 127.0.0.1). Click the blue shark fin icon in the top-left corner to start the capture. Wireshark will immediately begin capturing all traffic passing through that interface.

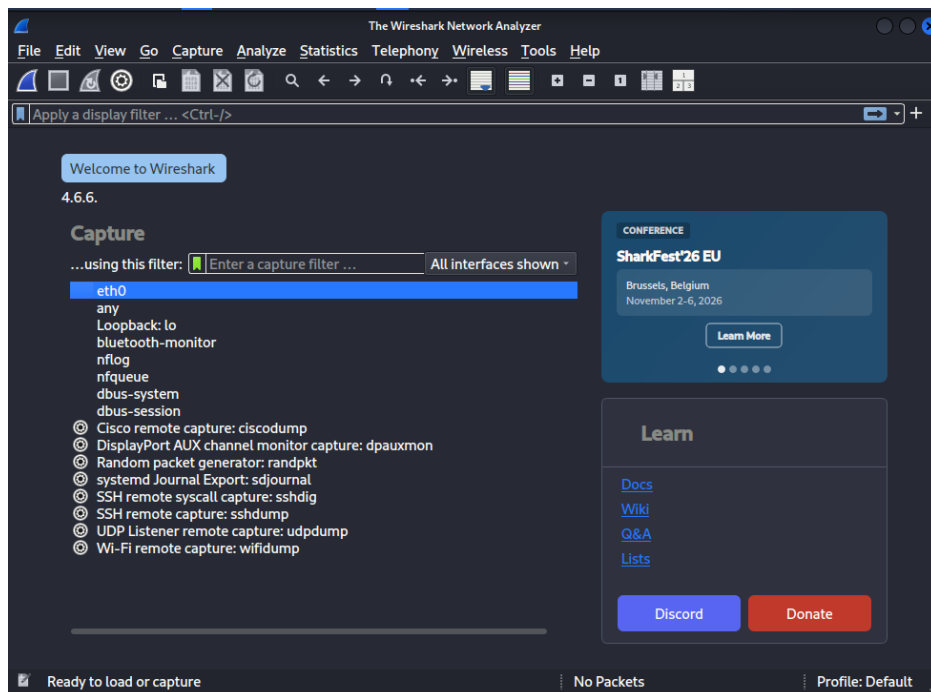


Fig 1: Wireshark interface selection screen – Loopback: lo selected.

Step 2: Generate Login Traffic

Open a web browser and navigate to the login page. Enter any dummy credentials. Click the login button. The login will fail, but the data has already been sent across the network.

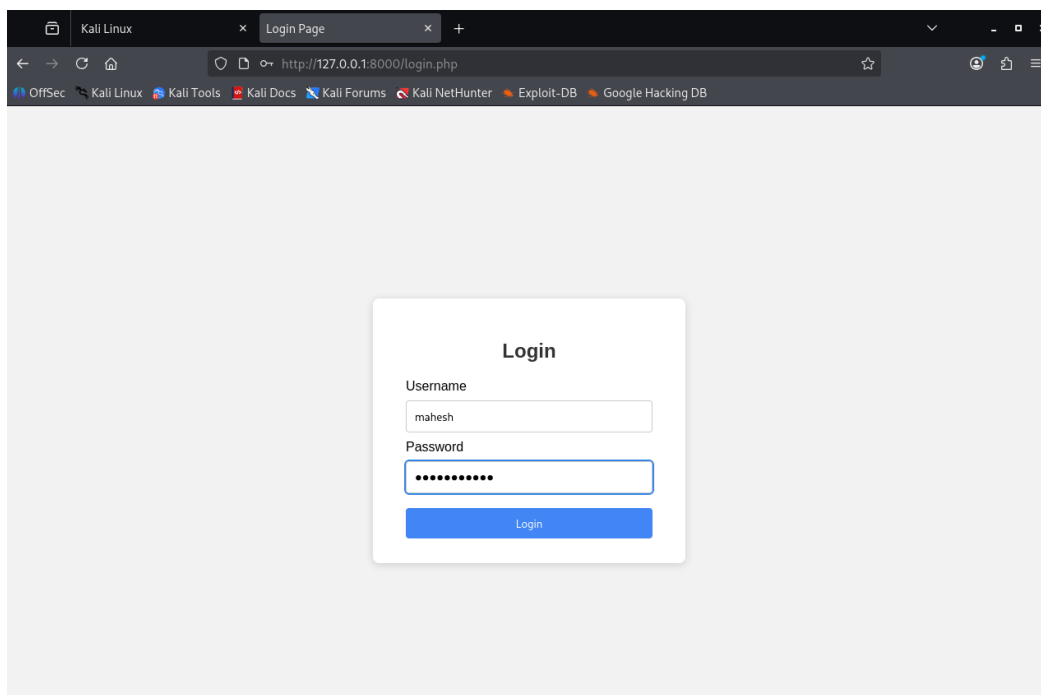


Fig 2: Login page with dummy credentials entered before clicking Login.

Step 3: Stop Capture and Filter Traffic

Return to Wireshark and click the Stop button (the red square). In the display filter bar, find the packet containing the login data. Since the form data was sent to the server, look for an HTTP POST request. Apply the following filter to find the exact packet and press Enter:

```
http.request.method == "POST"
```

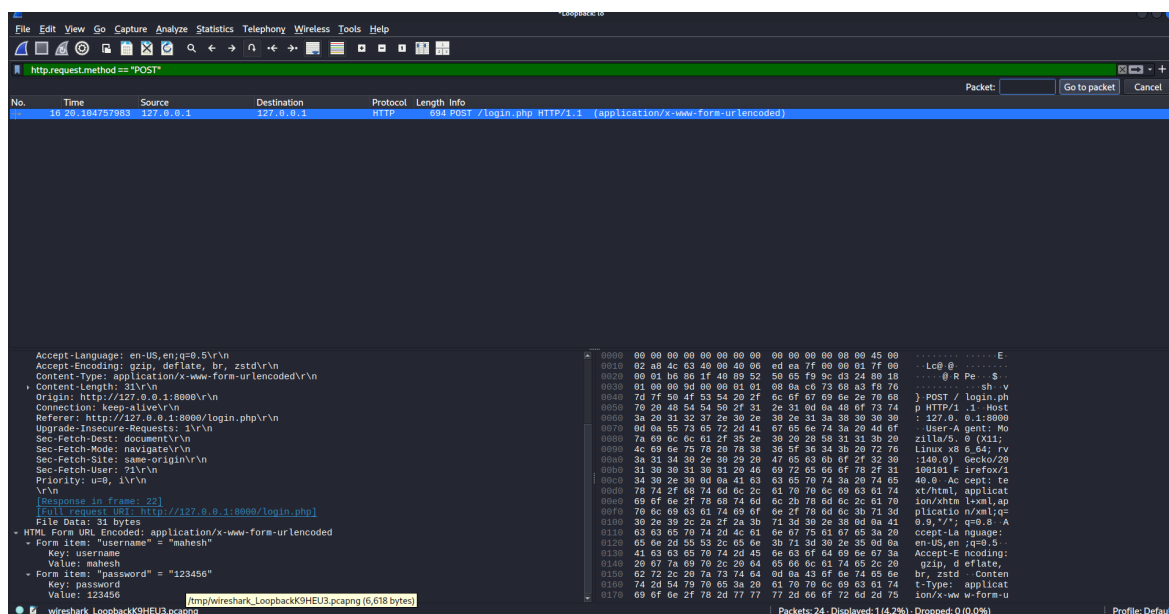


Fig 3: Filtered packet list showing the captured POST /login.php HTTP/1.1 request.

Step 4: Inspect the Packet to Find Credentials

In the filtered packet list, select the POST packet. In the Packet Details pane below the list, expand the following sections: Hypertext Transfer Protocol → HTML Form URL Encoded. Inside the

“HTML Form URL Encoded” section, the credentials entered will be visible in plaintext.

```
Wireshark - Packet 16 - Loopback: lo
> Frame 16: Packet, 694 bytes on wire (5552 bits), 694 bytes captured (5552 bits) on interface lo, id
> Ethernet II, Src: 00:00:00:00:00:00 (00:00:00:00:00:00), Dst: 00:00:00:00:00:00 (00:00:00:00:00:00)
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 46726, Dst Port: 8000, Seq: 1, Ack: 1, Len: 628
> Hypertext Transfer Protocol
> HTML Form URL Encoded: application/x-www-form-urlencoded

0190  6e 74 2d 4c 65 6e 67 74 68 3a 20 33 31 0d 0a 4f  nt-Lengt h: 31 0
01a0  72 69 67 69 6e 3a 20 68 74 74 70 3a 2f 2f 31 32  rigin: h ttp://12
01b0  37 2e 30 2e 30 2e 31 3a 38 30 30 30 0d 0a 43 6f  7.0.0.1: 8000--Co
01c0  6e 6e 65 63 74 69 6f 6e 3a 20 6b 65 65 70 2d 61  nnection : keep-a
01d0  6c 69 76 65 0d 0a 52 65 66 65 72 65 72 3a 20 68  live--Re ferer: h
01e0  74 74 70 3a 2f 2f 31 32 37 2e 30 2e 30 2e 31 3a  ttp://12 7.0.0.1:
01f0  38 30 30 30 2f 6a 65 67 69 6e 2e 70 68 70 0d 0a  8000/login.php--
0200  55 70 67 72 65 65 73 74 49 6e 73 65 63 75 72 65  Upgrade-Insecura
0210  2d 52 65 71 75 65 73 74 73 3a 20 31 0d 0a 53 65  -Request s: 1--Se
0220  63 2d 46 65 6e 74 0d 0a 44 65 73 74 3a 20 64 6f  c-Fetch- Dest: do
0230  63 75 6d 65 6e 74 0d 0a 53 65 63 2d 46 65 74 63  cumen-- Sec-Feto
0240  68 2d 4d 6f 6a 65 3a 20 6e 61 76 69 67 61 74 65  h-Mode: navigato
0250  0d 0a 53 65 63 2d 46 65 74 63 68 2d 53 69 74 65  : Sec-Fe tch-Site
0260  3a 20 73 61 6d 65 2d 6f 72 69 67 69 6e 0d 0a 53  : same-o rigin .S
0270  65 63 2d 46 65 74 63 68 2d 55 73 65 72 3a 20 3f  ec-Fetch -User: ?
0280  31 0d 0a 50 72 69 6f 72 69 74 79 3a 20 75 3d 30  1--Prior ity: u=0
0290  2c 20 69 0d 0a 0d 0a 75 63 65 72 6e 61 6d 65 3d  , i--u sername=
02a0  6d 61 68 65 73 68 26 70 61 73 73 77 6f 72 64 3d  mahesh&p assword=
02b0  31 32 33 34 35 36 123456
```

Fig 4: Packet Details pane showing the Hypertext Transfer Protocol section (collapsed).

```
Wireshark - Packet 16 - Loopback: lo
> Frame 16: Packet, 694 bytes on wire (5552 bits), 694 bytes captured (5552 bits) on interface lo,
> Ethernet II, Src: 00:00:00:00:00:00 (00:00:00:00:00:00), Dst: 00:00:00:00:00:00 (00:00:00:00:00:00)
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 46726, Dst Port: 8000, Seq: 1, Ack: 1, Len: 628
> Hypertext Transfer Protocol
> POST /login.php HTTP/1.1\r\n
> Host: 127.0.0.1:8000\r\n
> User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0\r\n
> Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n
> Accept-Language: en-US,en;q=0.5\r\n
> Accept-Encoding: gzip, deflate, br, zstd\r\n
> Content-Type: application/x-www-form-urlencoded\r\n
> Content-Length: 31\r\n
> Origin: http://127.0.0.1:8000\r\n
> Connection: keep-alive\r\n
> Referer: http://127.0.0.1:8000/login.php\r\n
> Upgrade-Insecure-Requests: 1\r\n
> Sec-Fetch-Dest: document\r\n
> Sec-Fetch-Mode: navigate\r\n
> Sec-Fetch-Site: same-origin\r\n
> Sec-Fetch-User: ?1\r\n
> Priority: u=0, i\r\n
> \r\n
[Response in frame: 22]
[Full request URL: http://127.0.0.1:8000/login.php]
File Data: 31 bytes
> HTML Form URL Encoded: application/x-www-form-urlencoded
> Form item: "username" = "mahesh"
Key: username
Value: mahesh
> Form item: "password" = "123456"

0260  3a 20 73 61 6d 65 2d 6f 72 69 67 69 6e 0d 0a 53  : same-o rigin .S
0270  65 63 2d 46 65 74 63 68 2d 55 73 65 72 3a 20 3f  ec-Fetch -User: ?
0280  31 0d 0a 50 72 69 6f 72 69 74 79 3a 20 75 3d 30  1--Prior ity: u=0
0290  2c 20 69 0d 0a 0d 0a 75 63 65 72 6e 61 6d 65 3d  , i--u sername=
02a0  6d 61 68 65 73 68 26 70 61 73 73 77 6f 72 64 3d  mahesh&p assword=
02b0  31 32 33 34 35 36 123456

Key (urlencoded-form.key), 8 bytes
Show packet bytes Layout: Vertical (Stacked)
```

Fig 5: HTML Form URL Encoded section expanded, revealing the credentials in plaintext.

Result

The experiment successfully intercepts the login credentials in a human-readable format. The analysis of the captured POST packet reveals the plaintext data that was transmitted over the network:

```
Form item: "username" = "mahesh"
Form item: "password" = "123456"
```

This result confirms the inherent security flaw of the HTTP protocol. Any sensitive data sent over HTTP is transmitted openly, making it trivial to intercept.